



Education

Lab 5.2. Argo Rollouts Blue-Green

Let's dig into it by creating a blue-green deployment scenario. It enables us to verify a version upgrade before the live traffic hits our service. It is easy to understand and therefore one of the most commonly used ways to roll out new versions of software without any downtime.

Objective

This lab aims to give a reader an idea of the “look and feel” of Argo Rollouts. It will demonstrate how to realize a simple blue-green scenario with Argo Rollouts. As blue-green is the most basic deployment pattern that rollout supports, this is a great introduction to the fundamental functionality of Argo Rollouts.

Prerequisites

- Kubernetes cluster with the argo-rollouts controller
- kubectl with the argo-rollouts plugin (optional)

Creating Blue-Green Deployments with Argo Rollouts

1. Install Resources

For the beginning, let's check for existing rollouts using this command:

```
$ kubectl get rollout
```

Output:

```
No resources found in default namespace.
```

As expected, there are no rollouts (yet) to be found in our cluster. Let's create one with the following command:

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: rollout-bluegreen
spec:
  replicas: 2
  revisionHistoryLimit: 2
  selector:
    matchLabels:
      app: rollout-bluegreen
  template:
    metadata:
      labels:
        app: rollout-bluegreen
    spec:
      containers:
      - name: rollouts-demo
        image: argoproj/rollouts-demo:blue
        imagePullPolicy: Always
        ports:
        - containerPort: 8080
  strategy:
    blueGreen:
      activeService: rollout-bluegreen-active
      previewService: rollout-bluegreen-preview
      autoPromotionEnabled: false
EOF
```

Output:

```
rollout.argoproj.io/rollout-bluegreen created
```

Check whether the Rollout resource has been created running the command below:

```
$ kubectl get rollout
```

Output:

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
rollout-bluegreen	2				75s

It has been created, but is not ready yet. Let's explore the reasons behind using the Argo Rollouts kubectl plugin to see if we can better understand its functionality.

The plugin enables us to get status information of a specific rollout and can be queried in the form: `kubectl argo rollouts get ro <rollout-name>`.

Command:

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

Output:

```
Name:          rollout-bluegreen
Namespace:     default
Status:        ✖ Degraded

Message:       InvalidSpec: The Rollout "rollout-bluegreen" is
invalid: spec.strategy.blueGreen.activeService: Invalid value:
"rollout-bluegreen-active": service "rollout-bluegreen-active" not
found

Strategy:     BlueGreen
Replicas:
  Desired:    2
  Current:    0
  Updated:    0
  Ready:      0
  Available:  0
```

NAME	KIND	STATUS	AGE	INFO
🕒 rollout-bluegreen	Rollout	✖ Degraded	9s	

The resource status is degraded, as not all requirements are met: If we look closely in our rollout manifest we see that we defined two services `activeService` and `previewService`. We need to make sure that the named services are available.

Let's create them with the command below:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Service
```

```
metadata:
  creationTimestamp: null
  labels:
    app: rollout-bluegreen-active
    name: rollout-bluegreen-active
spec:
  ports:
  - name: "80"
    port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: rollout-bluegreen
  type: ClusterIP
status:
  loadBalancer: {}
EOF
```

As mentioned, the rollouts resource references a second service. A so-called “preview” service. A preview service enables a preview stack to be reachable by an administrator. It does so without serving public traffic.

If we want the preview to go live, we need to “promote” the rollout. “Promotion” refers to setting a service live.

Therefore, we will create a preview service resource to be able to check our application before promotion (aka setting it live). Run the command below:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: null
  labels:
    app: rollout-bluegreen-preview
    name: rollout-bluegreen-preview
spec:
  ports:
  - name: "80"
    port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: rollout-bluegreen
  type: ClusterIP
```

```
status:
  loadBalancer: {}
EOF
```

If we now check our rollout, we will eventually see a **Healthy** status. Run the command below:

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

Output:

```
Name:          rollout-bluegreen
Namespace:     default
Status:        ✓ Healthy
Strategy:      BlueGreen
Images:        argoproj/rollouts-demo:blue (stable, active)
Replicas:
  Desired:     2
  Current:     2
  Updated:     2
  Ready:       2
  Available:   2
```

NAME	KIND	STATUS
AGE INFO		
○ rollout-bluegreen	Rollout	✓ Healthy
50s		
└─# revision:1		
└─□ rollout-bluegreen-5ffd47b8d4	ReplicaSet	✓ Healthy
14s stable,active		
└─□ rollout-bluegreen-5ffd47b8d4-mqc25	Pod	✓ Running
4s ready:1/1		
└─□ rollout-bluegreen-5ffd47b8d4-q4bgf	Pod	✓ Running
3s ready:1/1		

2. Perform an Upgrade

Now that we have a running application, let's try to perform a version upgrade using the blue-green method. Therefore, we'll adjust our image to deploy

argoproj/rollouts-demo:green instead of **blue**. Run the command below:

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: rollout-bluegreen
```

```
spec:
  replicas: 2
  revisionHistoryLimit: 2
  selector:
    matchLabels:
      app: rollout-bluegreen
  template:
    metadata:
      labels:
        app: rollout-bluegreen
    spec:
      containers:
        - name: rollouts-demo
          image: argoproj/rollouts-demo:green
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
  strategy:
    blueGreen:
      activeService: rollout-bluegreen-active
      previewService: rollout-bluegreen-preview
      autoPromotionEnabled: false
```

EOF

The Rollout status moves from “**Healthy**” to “**Paused**”, indicating that a rollout is in progress and waits for further action.

Please note, that we explicitly set `autoPromotionEnabled` to `false` - we can skip the pausing phase and directly promote by setting this value to `true`. Run the command below:

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

Output:

```
Name:          rollout-bluegreen
Namespace:     default
Status:        || Paused
Message:       BlueGreenPause
Strategy:      BlueGreen
Images:        argoproj/rollouts-demo:blue (stable, active)
               argoproj/rollouts-demo:green (preview)
Replicas:
  Desired:     2
  Current:     4
  Updated:     2
```

```
Ready:      2
Available:  2
```

NAME	KIND	STATUS
AGE INFO		
⌚ rollout-bluegreen	Rollout	Paused
91s		
—# revision:2		
└─□ rollout-bluegreen-75695867f	ReplicaSet	✓ Healthy
6s preview		
└─□ rollout-bluegreen-75695867f-m6pxh	Pod	✓ Running
6s ready:1/1		
└─□ rollout-bluegreen-75695867f-nr2rh	Pod	✓ Running
6s ready:1/1		
└─# revision:1		
└─□ rollout-bluegreen-5ffd47b8d4	ReplicaSet	✓ Healthy
55s stable,active		
└─□ rollout-bluegreen-5ffd47b8d4-mqc25	Pod	✓ Running
45s ready:1/1		
└─□ rollout-bluegreen-5ffd47b8d4-q4bgf	Pod	✓ Running
44s ready:1/1		

Let's investigate the rollout a little further and check replicaset with the command below:

```
$ kubectl get replicaset
```

Output:

NAME	DESIRED	CURRENT	READY	AGE
rollout-bluegreen-5ffd47b8d4	2	2	2	80s
rollout-bluegreen-75695867f	2	2	2	31s

Argo rollout created a second replicaset, which is used to manage the different pod versions. Lets promote the new version.

Command:

```
$ kubectl argo rollouts promote rollout-bluegreen
```

Output:

```
rollout 'rollout-bluegreen' promoted
```

Command:

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

Output:

```
Name:          rollout-bluegreen
Namespace:     default
Status:        ✓ Healthy
Strategy:      BlueGreen
Images:        argoproj/rollouts-demo:green (stable, active)
Replicas:
  Desired:     2
  Current:     2
  Updated:     2
  Ready:       2
  Available:   2
```

NAME	KIND	STATUS
AGE INFO		
○ rollout-bluegreen	Rollout	✓ Healthy
2m40s		
# revision:2		
└─□ rollout-bluegreen-75695867f	ReplicaSet	✓ Healthy
75s stable,active		
└─□ rollout-bluegreen-75695867f-m6pxh	Pod	✓ Running
75s ready:1/1		
└─□ rollout-bluegreen-75695867f-nr2rh	Pod	✓ Running
75s ready:1/1		
└─# revision:1		
└─□ rollout-bluegreen-5ffd47b8d4	ReplicaSet	•
ScaledDown 2m4s		
└─□ rollout-bluegreen-5ffd47b8d4-mqc25	Pod	○
Terminating 114s ready:1/1		
└─□ rollout-bluegreen-5ffd47b8d4-q4bgf	Pod	○
Terminating 113s ready:1/1		

Our new revision changed from “**preview**” to “**stable,active**” - indicating that the new revision is live.

You may also see that the first revision will display “**delay**” followed by a counter. Eventually, it will go into a “**ScaledDown**” status.

We can even see this by checking our service with the command below:

```
$ kubectl describe svc rollout-bluegreen-active
```

Output:


```
Name: rollout-bluegreen-active
Namespace: default
Labels: app=rollout-bluegreen-active
Annotations:
  argo-rollouts.argoproj.io/managed-by-rollouts: rollout-bluegreen
Selector:
  app=rollout-bluegreen,rollouts-pod-template-hash=75695867f
Type: ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.96.227.100
IPs: 10.96.227.100
Port: 80 80/TCP
TargetPort: 80/TCP
Endpoints: 10.244.0.43:80,10.244.0.44:80
Session Affinity: None
Internal Traffic Policy: Cluster
Events: <none>
```

Note that the Selector `rollouts-pod-template-hash` has the same value as the new ReplicaSet.

We just successfully performed a deployment using blue-green methodology.

3. Perform a Rollback

Let's assume we want to roll back from the new green to the old blue image.

Command:

```
$ kubectl argo rollouts undo rollout-bluegreen
```

Output:

```
rollout 'rollout-bluegreen' undo
```

Command:

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

Output:

```
Name: rollout-bluegreen
Namespace: default
Status: || Paused
Message: BlueGreenPause
Strategy: BlueGreen
```

```
Images:          argoproj/rollouts-demo:blue (preview)
                argoproj/rollouts-demo:green (stable, active)
```

```
Replicas:
  Desired:      2
  Current:      4
  Updated:      2
  Ready:        2
  Available:    2
```

NAME	KIND	STATUS
AGE INFO		
⊙ rollout-bluegreen	Rollout	Paused
3m52s		
—# revision:3		
└─□ rollout-bluegreen-5ffd47b8d4	ReplicaSet	✓ Healthy
3m16s preview		
└─□ rollout-bluegreen-5ffd47b8d4-2lvcq	Pod	✓ Running
3s ready:1/1		
└─□ rollout-bluegreen-5ffd47b8d4-k8sqf	Pod	✓ Running
3s ready:1/1		
—# revision:2		
└─□ rollout-bluegreen-75695867f	ReplicaSet	✓ Healthy
2m27s stable,active		
└─□ rollout-bluegreen-75695867f-m6pxh	Pod	✓ Running
2m27s ready:1/1		
└─□ rollout-bluegreen-75695867f-nr2rh	Pod	✓ Running
2m27s ready:1/1		

Note, that “undo” alone did not set the blue image active. The rollout is now again in the pausing phase, waiting for promotion of the rollout. Run the following command:

```
$ kubectl argo rollouts promote rollout-bluegreen
```

Output:

```
rollout 'rollout-bluegreen' promoted
```

Checking our rollout and active service once again, we see, that the selector changed back to our old ReplicaSet.

Command:

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

Output:

```

Name:          rollout-bluegreen
Namespace:     default
Status:        ✓ Healthy
Strategy:      BlueGreen
Images:        argoproj/rollouts-demo:blue (stable, active)
               argoproj/rollouts-demo:green

Replicas:
  Desired:     2
  Current:     4
  Updated:     2
  Ready:       2
  Available:   2

```

NAME	KIND	STATUS
AGE INFO		
🕒 rollout-bluegreen	Rollout	✓ Healthy
4m17s		
# revision:3		
└─□ rollout-bluegreen-5ffd47b8d4	ReplicaSet	✓ Healthy
3m41s stable,active		
└─□ rollout-bluegreen-5ffd47b8d4-2lvcq	Pod	✓ Running
28s ready:1/1		
└─□ rollout-bluegreen-5ffd47b8d4-k8sqf	Pod	✓ Running
28s ready:1/1		
# revision:2		
└─□ rollout-bluegreen-75695867f	ReplicaSet	✓ Healthy
2m52s delay:23s		
└─□ rollout-bluegreen-75695867f-m6pxh	Pod	✓ Running
2m52s ready:1/1		
└─□ rollout-bluegreen-75695867f-nr2rh	Pod	✓ Running
2m52s ready:1/1		

Command:

```
$ kubectl describe svc rollout-bluegreen-active
```

Output:

```

Name:          rollout-bluegreen-active
Namespace:     default
Labels:        app=rollout-bluegreen-active
Annotations:   argo-rollouts.argoproj.io/managed-by-rollouts: rollout-bluegreen
Selector:      app=rollout-bluegreen,rollouts-pod-template-hash=5ffd47b8d4

```

```
Type: ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.96.227.100
IPs: 10.96.227.100
Port: 80 80/TCP
TargetPort: 80/TCP
Endpoints: 10.244.0.45:80,10.244.0.46:80
Session Affinity: None
Internal Traffic Policy: Cluster
Events: <none>
```

4. Clean Up Resources

We successfully used the blue-green deployment pattern to deploy an application and even performed a rollback. To keep our working cluster nice and clean, we are going to clean up resources we created.

Command:

```
$ kubectl delete rollout rollout-bluegreen
```

Output:

```
rollout.argoproj.io "rollout-bluegreen" deleted
```

Command:

```
$ kubectl delete svc rollout-bluegreen-active
```

Output:

```
rollout-bluegreen-preview
service "rollout-bluegreen-active" deleted
service "rollout-bluegreen-preview" deleted
```